Computers & Security 154 (2025) 104438

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Enhancing intrusion detection in containerized services: Assessing machine learning models and an advanced representation for system call data

Iury Araujo [a,*], Marco Vieira [b]

[a] CISUC/LASI, DEI, University of Coimbra, Coimbra, Portugal
[b] University of North Carolina at Charlotte, Charlotte, USA

ARTICLE INFO

Keywords:
Intrusion detection
Machine learning
System calls
Graph-based representation

ABSTRACT

Security is one of the most critical requirements for modern digital systems. As the paradigm shifts from attempting to develop *fully* secure systems to designing resilient strategies that detect, respond to, and recover from attacks, Intrusion Detection Systems (IDS) become indispensable. However, developing robust IDS that address sophisticated attacks—especially in scenarios such as Cloud services, IoT, edge computing, and microservices, remains a significant challenge. Among these, containerized services present unique security challenges due to their architecture, deployment methods, and reliance on shared resources. On the other hand, Machine Learning (ML) offers promising, but not yet fully understood, solutions to enable automated, scalable, and adaptive intrusion detection mechanisms. In this paper, we study the applicability of a ML-based approach to enhance intrusion detection in containerized services by training and testing various ML algorithms on system call data, a commonly used data type in intrusion detection. Furthermore, we propose a novel graph-based representation for system calls that preserves critical relationships and contextual information between system calls. With this improved representation, we achieve enhancements in intrusion detection performance, including an increase in detection rates by at least 193% for the tested vulnerabilities while maintaining false alarms at a safer threshold, below a mean of 0.4% to maximize attack identification while minimizing false alarms we also incorporate a post-processing phase using a sliding window technique. This work not only addresses the challenges of securing containerized environments but also provides a robust framework for leveraging machine learning to build next-generation IDS.

1. Introduction

Security has become one of the most important, if not the most critical, requirement for any modern digital system, and the approach to developing security solutions has shifted significantly over time. Some security fields have explored alternative methods to secure systems in real time, recognizing that connected computational systems are constantly exposed to potential attacks and may never be entirely secure. As a result, the focus has moved to designing strategies to detect, respond to, and recover from attacks at runtime, ensuring that systems remain functional and operational (Kerman et al., 2020; Khraisat et al., 2019; Sarker et al., 2020).

The cornerstone of any digital security framework is an Intrusion Detection System (IDS). Without an IDS, it becomes impossible to identify ongoing attacks, deploy countermeasures, and mitigate their impact (Heidari and Jabraeil Jamali, 2023). Such systems are essential for safeguarding critical functionality and maintaining the integrity of systems in the face of ever-evolving threats (Khraisat et al., 2019).

Designing a robust IDS capable of addressing attacks that bypass known defense mechanisms or adapt to existing ones remains a challenging task across many scenarios, including Cloud services, IoT, edge computing, and microservices. Among these, containerized services stand out as a rapidly evolving domain, with efforts to integrate robustness-by-design principles into their detection systems (Cavalcanti et al., 2021). Containerized environments introduce unique security challenges due to their architecture, deployment methods, scalability, diverse anomaly types, and reliance on shared resources (Rocha et al., 2022; El Khairi et al., 2022).

Machine Learning (ML) has emerged as a promising solution for intrusion detection, and we consider that an ML-based IDS can tackle the challenges inherent to containerized services. The models developed through machine learning are typically lightweight after training and testing, making them well-suited for deployment in resource-constrained environments (Hamid et al., 2016; Sharma et al., 2019; Wang et al., 2019). Additionally, machine learning excels at identifying behavioral patterns through data analysis, which is particularly

* Corresponding author.
E-mail addresses: iuryrogerio@dei.uc.pt (I. Araujo), marco.vieira@charlotte.edu (M. Vieira).

https://doi.org/10.1016/j.cose.2025.104438
Received 20 December 2024; Received in revised form 10 February 2025; Accepted 11 March 2025
Available online 20 March 2025
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

advantageous in detecting sophisticated attacks, including zero-day vulnerabilities.

Although machine learning may be an essential component in developing next-generation intrusion detection systems for containerized services and beyond (Kiran et al., 2023; Sreelakshmi et al., 2024), ML techniques exhibit different behaviors depending on the scenarios they are exposed to. This way, more research is needed on evaluating which models are most suitable for each situation and understand how they react to various types of attacks. Furthermore, in the context of IDS, gaining deeper insights into the data and its underlying patterns is indispensable for developing effective detection models (Joraviya et al., 2024). Consequently, using the adequate data and a suitable representation that minimizes information loss are critical for enhancing the performance of IDS (Stewart et al., 2024).

In this paper, we present a study aimed at enhancing intrusion detection in containerized services by utilizing machine learning as the core solution. To achieve this, we conducted a set of experiments where we studied, trained, and tested a variety of machine learning techniques (e.g., Decision Tree, Random Forest, Isolation Forest, Elliptic Envelope, Local Outlier Factor, Multilayer Perceptron, etc.) to detect intrusions, leveraging system call data collected from the execution of containerized services (Araujo et al., 2023). The data was represented using a common approach for categorical data, specifically through label encoding, followed by a feature vector representation. Results show that machine learning techniques perform reasonably for intrusion detection using system call data.

Although system call data is commonly used in the intrusion detection domain (Grimmer et al., 2018; El Khairi et al., 2022; Aghaei and Serpen, 2019), our experiments show that common methods for representing categorical data are not well-suited for system calls. These methods often fail to capture the relationships and contextual information between system calls, thereby omitting critical details that could improve intrusion detection. To address this limitation, we developed a novel graph-based approach for representing system calls. The objective is to preserve the inherent information and relationships between system calls. Our approach is based on a comprehensive categorization of Linux system calls into groups and subgroups, building upon existing research in the field. Using this classification, we constructed a system call graph in which system calls are represented as nodes, and their relationships are represented as edges with varying weights (i.e., costs). Additional relationships were introduced based on the proximity of system calls, which we derived using insights from documentation and execution behavior.

With this new representation in place, we revisited our initial experiments and tested various machine-learning techniques using the graph-based representation. This enhanced representation led to significant improvements in detection, including a reduction in false positives (at least 30% for some techniques) and an increase in the ability to detect attack patterns detection rate (improved by at least 193% in all cases).

To further improve attack detection performance, we incorporated a post-processing phase by employing a sliding window technique to analyze the data. We selected a window size of 50 instances with an attack instance threshold set to 10% of the window size. The window slides using an overlapping technique, advancing by half the window size between consecutive checks. This approach maximized attack identification rates, with an average increase of approximately 55.4%, while further minimizing false alarms, with an average decrease of approximately 81.89%, which are critical for reducing the operational costs of security maintenance in containerized systems.

In summary, the contributions of this paper are as follows:

- A comprehensive evaluation of machine learning techniques through the training and testing of models for intrusion detection. In comparison to other studies on containerized services (El Khairi et al., 2022; Cavalcanti et al., 2021; Iacovazzi and Raza, 2022), this study covers a significantly broader range of techniques, exploring both supervised and unsupervised methods, as well as incorporating anomaly-based approaches and neural networks.
- A novel graph representation of system call data, enabling the transformation of categorical data into numerical data for machine learning while preserving the relationships within the system calls. This representation can be extended to research based on system call data across multiple scenarios, extending beyond the scope of this study.
- A study on the application of sliding window post-processing techniques to enhance the results of machine learning-based intrusion detection systems. This study, the first to use our novel representation, demonstrates the benefits of employing a sliding window approach with models trained on data with our representation.

The outline of this paper is as follows. Section 2 provides the necessary background and presents related work. Section 3 discusses the process of training and testing machine learning techniques for intrusion detection in containerized services. Section 4 introduces the novel representation for system call data, detailing its construction steps. Results are in Section 5, along with a discussion of their implications and relevance to our scenario. Threats to the validity of the work are discussed in Section 6. Finally, Section 7 summarizes our findings and provides concluding remarks.

2. Related work

Intrusion detection involves observing events within computer systems or networks and analyzing them for indications of security issues. An IDS identifies malicious actions and unauthorized access to safeguard computers and network infrastructures. It monitors and evaluates network traffic to detect unusual activities, notifying an administrator or automated system, which then determines the appropriate response (Abdulganiyu et al., 2023). An IDS must continuously monitor system activities, leveraging this data to differentiate between normal behavior and potential attacks.

2.1. Intrusion detection in containerized services

An IDS for containerized services must address data challenges introduced by the use of heterogeneous systems and technologies. System calls have long been a pivotal source of data, extensively employed in intrusion detection, malware analysis, and anomaly detection research for decades (Hofmeyr et al., 1998; Liu et al., 2018; Canfora et al., 2015; Castanhel et al., 2021; Forrest et al., 2008). These approaches leverage the inherent simplicity of system calls and their ease of acquisition in large volumes, making them a valuable and efficient resource for security analysis.

Recent studies have increasingly leveraged machine learning techniques with system call data to improve intrusion detection in containerized environments. For instance, El Khairi et al. (2022) introduced a novel anomaly-based Host Intrusion Detection System (HIDS) designed to tackle the unique challenges of these environments. By monitoring multiple properties of system calls and employing a graph-based model to analyze their dependencies within their context, this approach enhances the accuracy of distinguishing between normal and malicious activities. Evaluated on two datasets with 20 attack scenarios, it demonstrated superior performance over state-of-the-art tools, effectively detecting a wide range of anomalies with minimal runtime overhead. Similarly, Rocha et al. (2022) proposed a HIDS framework specifically tailored for Kubernetes container orchestration clusters. This framework employs system call analysis with machine learning within a distributed architecture, addressing scalability and resource overhead challenges. It generates alerts that provide actionable insights

for Security Operations Center (SOC) teams to respond to potential incidents, enhancing security in containerized environments.

Cavalcanti et al. (2021) examines the performance of container-level anomaly-based IDS in multi-tenant environments, where security concerns are heightened due to multiple containers sharing a single host. The study utilizes the Bag of System Calls (BoSC) technique combined with a sliding window approach and evaluates eight machine learning algorithms for classification. Results reveal that Decision Tree and Random Forest algorithms achieve the highest F-measure of 99.8% with a sliding window size of 30. However, Decision Tree outperforms Random Forest in terms of execution time, CPU usage, and memory consumption, making it a more efficient choice for real-time intrusion detection in containerized multi-tenant scenarios.

A graph-based representation is used in Iacovazzi and Raza (2022). The work tackles the challenges of intrusion detection in cloud container environments, where direct access to container resources is limited, and monitoring is complex. It introduces a novel system that uses system call data represented as graphs through the anonymous walks embedding algorithm to capture hidden relationships. This data feeds into a Random Forest classifier to differentiate container workloads and an Isolation Forest model to detect anomalies. Tested on two datasets, the approach demonstrates effectiveness in identifying malicious behaviors.

Limitations and our contributions: The works above present valuable ideas and solutions for Intrusion Detection Systems (IDS) in containerized environments. However, these approaches typically interpret system calls only in terms of their usage within the system and their sequential order. While this is undeniably important, we show in our work that system calls offer far more potential for analysis if we consider their categorization and the relationships they form before system execution. This broader perspective allows for creating representations that encompass all system calls, not just those executed during the detection period. Additionally, many of the existing studies focus narrowly on specific machine learning techniques, such as SVM, Decision Tree and Random Forest. Our work explores a wider range of methodologies that might yield better results.

2.2. System calls representation

Focusing on system call representation, some works provide valuable insights and innovative approaches, with graph theory serving as a significant source of inspiration. For example, Jang et al. (2015) developed a malware classification approach that applies social network analysis by transforming system call traces into system call graphs. By analyzing features such as degree distribution, degree centrality, average distance, clustering coefficient, network density, and component ratio, they successfully detected and classified various malware families. Similarly, Surendran et al. (2020) proposed a novel graph signal-based low-dimensional representation for system call analysis, specifically targeting sophisticated Android malware. Their approach reduced high dimensionality and incorporated system call dependency relations, integrating the features into a machine learning-based intelligent expert system for automated malware detection.

One of the most intriguing studies on system calls was conducted by Bagherzadeh et al. (2018). In their research, they analyzed a decade of Linux system calls to comprehend the changes that occurred throughout its lifecycle. They employed a classification method devised by Mauerer (2010) to categorize the system calls into ten groups and subsequently examined each group separately. Regrettably, they did not provide detailed information about this categorization or the specific system calls within each group. Additionally, the number of categories used seems relatively small for obtaining a deeper understanding of the inherent behavior among system calls.

Limitations and our contributions: These works demonstrate that, despite over two decades of using system calls for monitoring and analysis, the categorization and representation of system calls have not received significant research attention. While some noteworthy examples of innovative representations exist, most are highly specific to particular scenarios and cannot be easily generalized to the broader set of Linux system calls. However, a particularly inspiring finding was the use of graph-based solutions for representing system calls. Although these approaches were also tailored to specific contexts, their underlying concepts and methodologies provided valuable inspiration for our work. We firmly believe in the potential to create an enhanced representation of system calls through a graph-based approach, one that captures the relationships between system calls and offers greater generality for application across diverse scenarios, surpassing the existing options in the literature.

3. ML-based intrusion detection evaluation

To study the effectiveness of machine learning-based intrusion detection in containerized services, we started by designing a set of experiments with various ML techniques (let us name it as *preliminary study*). These experiments provided insights on the capabilities of such solutions, but also revealed shortcomings in the representation of system call data. Specifically, the representation failed to retain sufficient information to build effective models. This challenge was further compounded by the imbalanced nature of the training data, which made the task significantly more complex.

In an attempt to achieve a more robust representation of the data, we developed a graph-based representation for system calls and introduced a sliding window technique to analyze the data. With this novel solution, we conducted a second set of experiments (let us name it as *extended study*), including not only techniques used in the preliminary study but also other machine-learning techniques that are incompatible with the traditional representations of system call data.

In this section we discuss the processes behind the preliminary study and the extended study. We start by discussing the data collection and then present details on the preliminary study, which are followed by a discussion about the extended study. We then introduce the machine learning techniques study and the evaluation metrics used to characterize those techniques. Note that, to ensure clarity and thorough understanding, the methodology used to create and validate the system call representation is detailed in a dedicated section (Section 4).

All the experiments were implemented in Python 3.8.10. The free library Scikit-learn version 0.24.2 was used for ML training and testing. All the phases and tasks of the evaluation procedure were conducted in a machine with the following specifications: Intel Core i5 10400 CPU at 2.90 GHz with 32 GB of RAM, with Ubuntu 20.04.2 LTS.

3.1. Data collection

The data utilized for training and testing the ML techniques in both preliminary and extended studies was derived from the work conducted by Flora et al. (2020), which focuses on assessing the effectiveness of intrusion detection in cloud container-based systems. Their research considered a MariaDB setup on a Docker container to emulate a service. Using the TPC-C benchmark, they applied a dynamic workload to generate requests to the container. This workload involved varying numbers of active clients, beginning with ten active connections, gradually increasing to a peak of 90 connections, and decreasing back to ten active connections. Furthermore, the workload periodically executed commands simulating administrative operations. Due to its inherent compatibility with containerized environments, the *sysdig* tool was used to monitor system calls.

The data was organized into collections, where each collection comprises system calls executed by the container over a specific time frame (30 min). At the midpoint of the collection period (15 min), an attack on the MariaDB instance was initiated. The ending time of each attack varies depending on the vulnerability exploited and the system workload at the time of the attack. After the attack completes, the

Table 1
List of vulnerabilities exploited in this work. Flora et al. (2020).

CVE ID	Acess	Vulnerability type(s)	CVSSS	ID
CVE-2016–6662	Remote	Execute code, Bypass	10.0	V1
CVE-2012–5611	Remote	Execute code, Overflow	6.5	V2
CVE-2013–1861	Remote	Denial of service, Overflow	5.0	V3
CVE-2012–5627	Remote	Execute code	4.0	V4
CVE-2016–6663	Local	Gain privileges	4.4	V5

Table 2
Proportion of normal and attack instances in the training and testing data.

Vulnerability	Instances	Normal instances	Attack instances
V1	567 498	565 803 (99.7013%)	1695 (0.2987%)
V2	501 380	565 803 (99.7013%)	469 (0.0935%)
V3	492 055	491 276 (99.8417%)	779 (0.1583%)
V4	642 561	554 446 (86.2869%)	88 115 (13.7131%)
V5	565 299	562 120 (99.4376%)	3179 (0.5624%)

end time is recorded, which allows dividing each collection into three phases: pre-attack, attack, and post-attack (more details in Section 3.5). Five distinct vulnerabilities were exploited to generate attack data for training and testing the ML models. These vulnerabilities were associated with an older MariaDB version, with proof-of-concept code obtained from *exploitdb.com*. For each vulnerability, three collections were created. Table 1 summarizes the selected vulnerabilities and their identification in this study. Additionally, three collections were generated containing only benign data gathered over an extended duration ($24H$).

The containers for the collection of data in the work by Flora et al. (2020) consisted of Ubuntu 14.04 LTS and MariaDB 5.5.28 with default configurations. Two machines were used to run the containers, a Target Infrastructure and a Test Driver. The target infrastructure machine run the evaluated service and had a 6-core 2.8 GHz CPU with 32 GB of RAM, with Ubuntu 18.04.2 LTS. The Test Driver, in charge of driving the experiments, ws based on a 3.20 GHz CPU and 16 GB of RAM, with Ubuntu 16.04.3 LTS.

3.2. Preliminary study design

In this study, we employed a generic approach to handle categorical data by utilizing label encoding, followed by data separation and transformation into feature vectors. The system call data, categorical by nature, is described as distinct groups or categories based on specific attributes or characteristics. As such, it is qualitative rather than numerical, usefull to label features rather than to measure quantifiable amounts (Hancock and Khoshgoftaar, 2020). Since machine learning algorithms cannot process raw categorical data in this format, it had to be transformed into a numerical representation that the algorithms could understand.

System call data typically consists of the system call's name, its parameters, and the timestamp of the invocation. However, research often excludes parameters due to the complexity and cost of standardizing and converting this information (Wunderlich et al., 2020). Therefore, we also focused solely on the system call name and timestamp. The names of system calls were first converted into numeric values using a simple dictionary, and each system call was assigned a unique numerical identifier. Next, the sequence of system calls was divided chronologically into instances, each containing 50 system calls. This number was chosen after a series of tests with varying instance sizes $n = \{25, 50, 100, 200, 400\}$. We analyzed the True Positive Rate (TPR) during the attack phase, aiming for the highest results while maintaining a low False Positive Rate (FPR) during the pre-attack and post-attack phases. The comparison revealed that an instance size of 50 provided the best balance among the tested values and was, therefore, selected as the default configuration. All results from this analysis can be found in: https://figshare.com/s/6152d90559dac565f248.

After creating the instances, we processed the data to reduce its dimensionality and organize it using feature vectors. This approach is conceptually similar to the "Bag of System Calls" model, which is modeled after the concept of the Bag of Words (Abed et al., 2015), but with an important distinction: since we knew all possible system calls present in our dataset, we created a vector where each position corresponded to a specific system call type. Even if a particular system call was not present in a given frame, its position in the vector was maintained with a value of zero. This ensured a consistent representation across all instances.

During the data representation process for the system call data, we analyzed the distribution of instances across the two classes, normal and attack, and identified a significant imbalance. This imbalance was especially pronounced for attacks exploiting vulnerabilities V1, V2, V3, and V5. Table 2 illustrates this imbalance by showing the number of instances for each class, revealing that attacks using these vulnerabilities account for less than 1% of the data. Addressing this imbalance was critical in designing and evaluating various configurations for our ML techniques.

3.3. Extended study design

For the extended study, we utilized our novel graph-based representation for system call data, as detailed in Section 4, which is intentionally designed to be highly flexible, enabling researchers to export and transform it into various formats for diverse applications. In this study we also included a post-processing sliding window to enhance the detection results.

As the raw graph representation cannot be directly used with ML techniques, an additional processing step is required: transforming the graph nodes into vector representations within a vector space. To transform the nodes into vectors, various studies offer potential approaches (Perozzi et al., 2014; Grover and Leskovec, 2016; Hamilton et al., 2017; Ribeiro et al., 2017). For this work, we employed the Node2vec algorithm, as introduced by Grover and Leskovec (2016).

Node2vec is a machine learning algorithm specifically designed to learn continuous feature representations, or embeddings, of nodes in a graph. These embeddings effectively capture both the structural and relational information of the nodes within the graph. We found Node2vec to be the most suitable approach for our work due to its flexibility in exploring node connections and its ability to capture both homophily (nodes with similar attributes being closely embedded) and structural equivalence (nodes with similar roles in the graph, regardless of their direct connections).

To use Node2vec, we first generated an adjacency matrix of our graph. We opted for the algorithm's default dimensionality of 128 for the embeddings. The output of the algorithm is a 128-dimensional embedding for each node, representing the node in a vector space. Using these embeddings, we transformed the system calls into their corresponding vector representations, with each dimension of the embedding serving as a feature for the ML algorithms. This representation eliminates the need to segment system calls into frames (which was required for the preliminary study). While the framing approach provides more information for training, it significantly increases the number of instances by approximately 50 times, introducing, unfortunately, the need for more computational resources.

Post-processing using a sliding window: Post-processing leverages the outputs from the tests of the extended evaluation study, which consist of sequences of system calls, each one classified with a binary value $f = \{0 - Normal, 1 - Attack\}$. As the system calls are arranged in chronological order, preserving the exact sequence of events as they were originally collected, we utilize a window of size n and define an attack threshold percentage p. The window examines n syscalls in each

iteration, determining how many are classified as attacks. If the number of attack-classified syscalls meets or exceeds $x = ((n * p) \div 100)$, an attack alert is triggered. For example, with a window size $n = 100$ and a threshold $p = 20\%$, each iteration evaluates 100 syscalls. If twenty or more syscalls are classified as attacks, the method triggers an attack alert at that point in the timeline.

For a given run, the sliding window process begins with the first set of system calls in the timeline. After determining whether an alert should be triggered, the window shifts. Based on our evaluation, shifting the window by half its size yields good results (other sizes can be considered depending on the scenario). The window then re-evaluates the subsequent n system calls. This is repeated until there are no longer enough system calls to fill the window size.

The sliding window design is intended to improve detection during the attack phase while reducing false positives during the pre- and post-attack phases. By carefully adjusting the shift size and window parameters, the approach balances sensitivity and false alarm rates, ensuring more reliable performance across different timeline segments. Based on extensive testing conducted for this scenario and dataset, a window size of $n = 50$ with a threshold of $p = 10\%$ was determined to be the most effective configuration for the post-processing approach.

3.4. Machine learning techniques

We selected a diverse range of ML techniques, supervised and unsupervised. Some are commonly used in the context of intrusion detection, while others are more prevalent in anomaly detection (Sreelakshmi et al., 2024; Tripathy and Behera, 2023). An analysis of the system call data showed a significant class imbalance in attacks targeting four of the five tested vulnerabilities (see Table 2). This imbalance heavily influenced the configuration of our techniques, as addressing it was essential for accurately evaluate performance. Consequently, we applied pre-processing steps involving undersampling, oversampling, or a combination of both.

The selected ML algorithms are: Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), Isolation Forest (IF), One-Class SVM, Elliptical Envelope (EE), Local Outlier Factor (LOF), Complement Naive Bayes (CNB), and Multilayer Perceptron (MLP). We selected these because they are among the most common techniques used in ML-based intrusion detection, particularly in areas such as cloud computing and IoT (Balyan et al., 2023).

A set of configurations was defined for each algorithm. For DT, RF, IF, EE, LOF and CNB, the first configuration used default settings without any modifications (i.e., no specific configuration was provided as parameters, and Scikit-learn used only the default settings predefined in the algorithms), and the third and fourth configurations employed pre-processing techniques to address class imbalance: oversampling to enhance minority class representation and undersampling to reduce majority class dominance. The second configuration was tailored to each technique to better address specific challenges: for DT and RF, class weights were applied using the balanced configuration, calculated as: $Weight_j = n\_instances \div (n\_classes * n\_instances_j)$, where j represents each class; for IF, EE, and LOF, the contamination parameter was adjusted to reflect the normalized proportion of attack instances, ensuring it fell within the range of 0 to 1; and for CNB, the norm parameter was set to True, enabling an additional normalization step on the weights to improve the handling of class imbalance.

For SVM and One-Class SVM, each configuration utilized a distinct kernel to evaluate its impact on performance. Since both techniques tend to be less effective with large datasets, all configurations incorporated undersampling to manage dataset size and enhance computational efficiency. The configurations tested various kernel types, specifically Linear, Polynomial (Poly), Radial Basis Function (RBF), and Sigmoid, in this order.

For MLP, there were some differences in the configurations compared to the other techniques. Configurations one, three, and four followed the same pattern as before: the first configuration used default parameters without modifications, while the third and fourth applied pre-processing techniques to address class imbalance using oversampling and undersampling, respectively. However, in the second configuration, we modified the solver parameter, which determines the algorithm for weight optimization, changing it from the default to lbfgs.

Additionally, unlike the other techniques, we aimed to investigate how increasing the number of hidden layers could improve detection rates. To achieve this, we introduced four additional configurations specifically for MLP. For each of these configurations, we incremented the number of hidden layers by one and halved the number of neurons in each subsequent layer. Starting with the number of features as the size of the first layer, the structure progressively reduced in size. For example, if the dataset had 100 features, the hidden layers in the seventh configuration would be structured with neurons as follows: $\{100, 50, 25, \}$.

3.5. Evaluation and metrics

To conduct both the preliminary and extended studies, we set aside a portion of the data to evaluate the model's generalization. As previously explained, each attack, executed through a specific vulnerability, generated three separate data collections. For each evaluation, one of these collections was exclusively reserved for testing, while the remaining two were used for training. This process was repeated three times, with a different collection designated for testing in each iteration. For example, if we aim to detect a vulnerability V1, we would have three collections of data containing records of attacks, labeled as C1, C2, and C3. These collections would be split as follows: (Training: C1, C2 | Testing: C3), (Training: C1, C3 | Testing: C2), (Training: C2, C3 | Testing: C1). This approach ensures that the model is tested on previously unseen data in each iteration.

To run the preliminary and extended studies, we set aside a portion of the data to test the model's generalization. As previously explained, each attack, executed through a specific vulnerability, produced three data collections. For each evaluation, one of these collections was exclusively reserved for validation, while the remaining two were used for training. This process was repeated three times, with a different collection designated for testing in each iteration.

We included a supplementary collection of baseline data, gathered over an extended period during normal container operations. This was necessary because, in our initial attempts, we observed a high false alarm rate. Thus, adding more normal data during training helps the models better identify normal patterns, thereby reducing the misclassification of normal instances. Addressing this issue was crucial, as intrusion detection systems must strike a delicate balance between effectively detecting intrusions and minimizing false alarms. High false alarm rates can be problematic due to the costs associated with deploying countermeasures. Excessive false alarms complicate system operation and can undermine the practicality of the system (Laszka et al., 2016).

This study focuses on two key metrics: sensitivity (True Positive Rate, TPR) and false alarm ratio (False Positive Rate, FPR). These metrics are derived from the confusion matrix during the testing phase. The choice of these metrics was guided by the need to evaluate the TPR during the attack phase and the FPR in the pre-attack and post-attack periods. As mentioned before, to enhance the evaluation process, the results were divided into three distinct periods: pre-attack, attack, and post-attack. Fig. 1 illustrates the timeline of the testing data. For a model to be deemed successful, it should demonstrate a high TPR during the attack phase while maintaining a low FPR during both the pre-attack and post-attack phases.

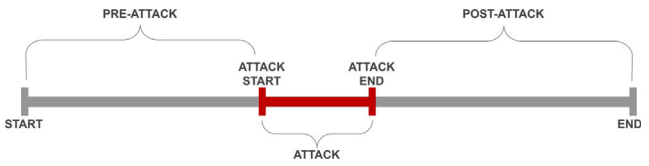


Fig. 1. Division of the testing data into three time periods.

4. System call classification and graph-representation

System calls are a valuable, real-time source of system activity data, widely used in anomaly detection systems (Kim et al., 2016). Typically, they include execution time, call name, and parameters, though monitoring systems primarily focus on the timestamp and call name. The sequence of system calls also offers critical insights into system behavior.

System call data, being categorical, requires transformation for effective use. The challenge lies in the fact that existing representations are not specifically designed for system calls, resulting in the loss of inherent information regarding the interaction between different system calls. For example, the syscalls *read*() and *write*() have different effects in a file. Nevertheless, they have inherent information that can be identified because they use the same resources to do their opposites tasks, thus having a connection between them. These relationships can provide valuable insights for system analysis. To address this, we propose a robust representation that preserves these interactions, enhancing the usability and value of system call data for various applications.

To ensure the representation's applicability across various scenarios involving system calls, we decided to base it on graph theory. By employing an undirected graph, we developed a representation that captures the core characteristics of system calls. In this context, nodes represent individual system calls, while edges, assigned with varying costs, represent diverse relationships between them.

Fig. 2 shows the sequential development of our enhanced system call representation. First, system calls were categorized into classes and subclasses to form a foundational framework for their relationships within the graph. This process was initially based on existing literature and was refined with additional insights. Categories and subclasses were determined by three researchers, with disputes resolved by an independent group to ensure accuracy and consistency. It is important to clarify that throughout the representation work, we use the terms categories and classes, as well as subcategories and subclasses, interchangeably.

Following categorization, a system call graph was constructed by analyzing Linux system call documentation to identify relationships based on tasks, resources, and parameters. Additional relationships were marked as custom, as shown in Fig. 2. Recognizing the potential for subjectivity, a validation process was introduced, consisting of two stages: validating categorization and verifying graph relationships. The final representation combines categorization, representation, and validation.

The scope was limited to Linux Kernel system calls, a common focus in monitoring and analysis literature. At the time of this work, 462 system calls (Linux Kernel 5.16) were available, but deprecated, obsolete, or architecture-specific calls were excluded, leaving 405 calls in the final representation. Updates for new or modified system calls can be easily incorporated using the outlined methods. All data and artifacts are accessible at: https://figshare.com/s/6152d90559dac565f248..

4.1. Categorization

The first step in enhancing system call representation was categorizing them into classes to establish relationships and facilitate analysis. Following the classification in Operating System Concepts (Silberschatz et al., 2018), six categories were defined: Process Control (e.g., creating and scheduling processes), File Management (e.g., file operations and directory management), Device Management (e.g., interacting with hardware devices), Information Maintenance (e.g., retrieving system configurations), Communications (e.g., inter-process communication), and Protection (e.g., managing access permissions and enforcing security).

System call documentation was analyzed to assign categories. Based on definitions in Silberschatz et al. (2018), some system calls were placed in multiple categories to preserve information and enhance graph relationships. For example, Linux syscalls *read*() and *write*() were classified under File Management and Device Management, as they interact with both files and devices.

The categorization divided system calls into six categories. However, the diversity of tasks and resources within categories, such as the 146 syscalls in File Maintenance, posed challenges for relationship analysis. To address this, we introduced an additional grouping within each category. This clustering created subcategories of system calls with similar purposes, functions, or resource usage, making their relationships more manageable and recognizable.

Because some system calls belong to more than one class, when creating these subclasses, we had to consider that two classes would share some of these subclasses. Therefore, when clustering the system calls together, some previous system calls with only one class were adapted to have two classes as the others in their group. For instance, initially, the system calls *capget*(2) and *capset*(2) were classified solely under Process Control. However, when the subcategories were created, they were grouped together under Thread Management, along with other system calls like *getcpu*(2) and *gettid*(2), resulting in a dual categorization under both Process Control and Information Maintenance. As a result, *capget*(2) and *capset*(2) now possess this dual categorization as well. Also, some system calls were so difficult to cluster that they created their own subclass (e.g., *bpf*(2), *reboot*(2) and *seccomp*(2)).

Table 3 shows the 66 subclasses created, with 54 belonging to a single class (e.g., Extended Attribute Management, Process Creation, Permission Management, Memory Protection) and 12 shared between two classes. For instance, Filesystem Management is shared by File Management and Information Maintenance, while Inter-Process Communication is shared by Process Control and Communications. All the classes and subclasses and information about the classification are available at: https://figshare.com/s/6152d90559dac565f248.

Table 3
Number of subcategories for every single category. Total = Subcategories (Shared subcategories).

	Process Control	File Management	Device Management	Information Maintenance	Communications	Protection	Total
Process Control	15	1	0	7	1	0	24 (9)
File Management	1	15	1	1	0	0	18 (3)
Device Management	0	1	2	0	0	0	3 (1)
Information Maintenance	7	1	0	13	0	0	21 (8)
Communications	1	0	0	0	4	1	6 (2)
Protection	0	0	0	0	1	5	6 (1)
Total	24 (9)	18 (3)	3 (1)	21 (8)	6 (2)	6 (1)	78 (12)

4.2. Representation

The second step in enhancing the system call representation involved translating the classification into a system call graph. As shown in Fig. 3, two strategies can be considered: define classes and subclasses only as edges of different costs connecting all the nodes directly or create nodes specifically to represent the classes and subclasses in the graph and attach the system call nodes to them. We chose the latter, despite longer path lengths, for two key reasons. First, it allows easier reconfiguration, as system call nodes can be added or removed without affecting all connections. Second, class and subclass nodes act

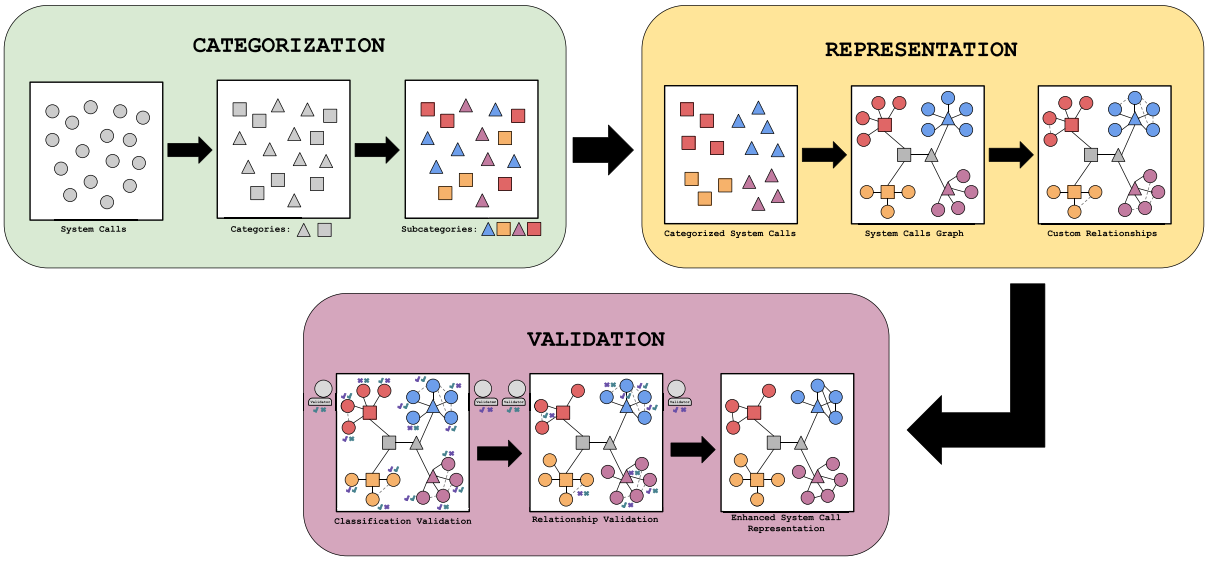


Fig. 2. Sequential development of our enhanced system call representation using graph theory.

Table 4
System calls relationships.

Relationship	Edge cost	Cost formula	Explanation
ClassToClass	R	$4 * T$	A relationship formed between two categories of system calls. This represents the most costly path in the graph, highlighting the distinctions between system calls from different categories.
SharedClassToClass	S	$2 * T$	A relationship established between categories that share a subcategory. This results in a shorter path between them due to certain system calls being linked to both categories.
ClassToSubclass	T	T or 1	A relationship established between a category and its subcategory. This connection signifies the most straightforward path in the graph and is utilized as the basis for calculating the cost of all other relationships.
SyscallToSubclass	U	$T/2$	A relationship established between a system call and its subcategory. The cost associated with this edge reflects the proximity of the system call to its subcategory, constituting the smallest cost designed from the classification process.
SyscallToSyscall	V	$T/4$	A relationship forged between system calls that exhibit similarities in their execution or interaction with the same resources. This connection is characterized by a more comprehensive analysis of the system call documentation.
SyscallToSyscallAnalogous	X	$T/8$	A relationship established between system calls that are highly analogous, differing primarily in the number of parameters or the system where they are employed, while their core functionality remains identical.

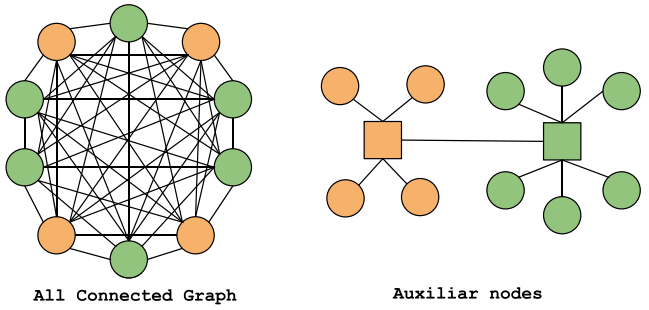


Fig. 3. Two strategies for constructing the system call graph are exemplified: either using an all-connected graph approach or employing auxiliary nodes to represent categories and subcategories.

as centroids, facilitating transformations like converting the graph into vector space while providing flexibility for researchers.

We propose six distinct types of relationships among system calls, namely: ClassToClass, SharedClassToClass, ClassToSubclass, SyscallToSubclass, SyscallToSyscall, and SyscallToSycallAnalogous. These relationships are denoted by different edge types within the graph, each of which carries its unique associated costs. Specifically, these edge costs are represented as R, S, T, U, V, and X, respectively. Table 4 presents comprehensive information concerning these relationships, along with their corresponding associated costs. It is important to note that these costs can be easily adjusted based on the domain or scenarios in which the system call graph is being utilized.

Edges in the graph are assigned costs based on relationships. R Cost (ClassToClass), the cost between unrelated classes, is ($R = 4 * T$), emphasizing the distance between system calls in different classes. S Cost (SharedClassToClass) connects classes sharing subclasses and has a cost of ($S = R/2 = 2 * T$), reflecting closer relationships. T Cost (ClassToSubclass), the connection between classes and their subclasses, is the base unit of cost, T, which users can adjust as needed, though a default of ($T = 1$) is suggested. U Cost (SyscallToSubclass) represents connections between system calls and their subclass, valued at $U = T/2$, ensuring the path length between system calls within the same

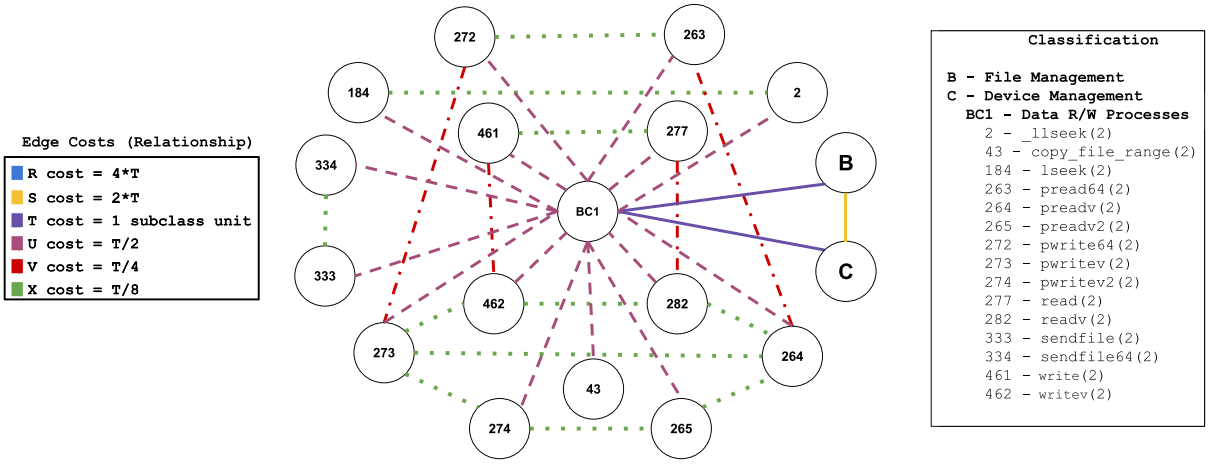


Fig. 4. A example from the system call graph. It presents the subclass Data R/W Processes, their system calls, and relationships.

subclass is no more than T. This flexible structure supports diverse graph analyses.

Fig. 4 shows the subclass Data R/W Processes shared by the classes File Management and Device Management. They are represented as the BC1, B, and C nodes, respectively. This figure shows a small part of the system call graph created to enhance the representation of system calls. It is possible to see in the figure the system call nodes and how the relationships are kept intact to be used by any analysis process. However, the edge costs explained until now were created based on our classification of the system calls, but they are not the only ones that can be established between system calls. We also worked to represent relationships based on the closeness between their tasks, resources, and scope, thus the need for defining the relationships SyscallToSyscall and SyscallToSyscallAnalogous with cost V and X, respectively.

Edges with costs $V = T/4$ and $X = T/8$ are not based on system call categorization but reflect task and resource similarities. V Cost (SyscallToSyscall) represents relationships between system calls with related tasks or resource usage, identified through documentation analysis. For example, in Fig. 4, a V cost edge connects *read*() (277) and *readv*() (282) due to similar tasks but differing resource usage and responses. V costs also link calls with distinct tasks sharing a common input/output purpose.

X Cost (SyscallToSyscallAnalogous) represents relationships between system calls that perform the same function with minor differences, such as an added parameter or version update. For instance, *pwritev*() (273) and *pwritev2*() (274) share an X cost edge, as pwritev2() includes an additional fifth parameter.

The final system call graph consists of 477 nodes, of which 405 are system calls, 66 are subcategories, and 6 are categories. This graph comprises 1427 edges, categorized into R (9 edges), S (6 edges), T (726 edges), U (405 edges), V (193 edges), and X (88 edges) costs. While these numbers are final for the current representation, it is worth noting that they may change in the future as new system calls are added or removed from the kernel.

4.3. Validation

The categorization and representation processes relied on documentation and literature but were influenced by the authors' perceptions, introducing potential subjectivity. To mitigate this, we implemented a two-step validation: first, validating the categorization, and second, verifying the relationships.

For validation, we sought external evaluation from researchers experienced in using system calls. To identify suitable candidates, we analyzed proceedings from key security and dependability conferences (2018–2022), including USENIX Security, DSN, and IEEE Security and Privacy, identifying 212 authors focused on system calls. Researchers were randomly selected from this list and sent relevant information and a questionnaire via email, with a one-month response period. If no response was received, new validators were contacted until the list was exhausted. Unfortunately, the number of responses was insufficient for a double review of all questionnaires.

4.3.1. Categorization validation

To validate our categorization process, we created a control group of 54 easily categorizable system calls, such as *accept*(), *connect*(), and *open*(), which did not belong to shared subcategories. The remaining 405 system calls were divided into sets of 15, with each set containing two calls from the control group and the rest selected randomly. The control group assessed the validator's effort and knowledge, with responses discarded if control group classifications were incorrect. This ensured the reliability of the categorization process.

For each set, we created a questionnaire where each system call was transformed into a related question. Validators were asked to categorize each system call using the available classes and subclasses, with brief explanations provided to avoid confusion. While some system calls could belong to two classes, validators were limited to selecting one, enabling us to analyze the correctness of dual categorizations. This analysis considered not only the validation responses but also other system calls within the same subcategory. A total of 27 questionnaires were created, with two validators assigned to each, ideally providing a double review for every system call.

Fig. 5 shows the results of the first validation. We received 32 responses, covering 48% of system calls with a double review and 81% with at least one review. Validators' categorizations were compared to our own, following defined rules. Responses failing to match our control group categorizations were discarded. If at least one validator's assessment agreed with ours, the original categorization was retained. When both validators agreed on a different categorization, we updated it accordingly. For disagreements between validators and our team, we analyzed responses within the same subcategory to make a more informed decision, as illustrated in Fig. 6.

In the end, 67% of system calls matched our categorization and the validators'. For the 14% that differed, we resolved conflicts case by case, changing 7% to match the validators and retaining 5% after evaluating related system calls in the same subclass. The remaining 2% were unchanged due to inaccurate validator responses. Of the 19% unvalidated system calls, 11% were adjusted by analyzing patterns in validators' feedback.

During the analysis of differing categorizations, patterns emerged showing validators classifying some system calls in the same subcategories into different categories. This prompted the consolidation of six

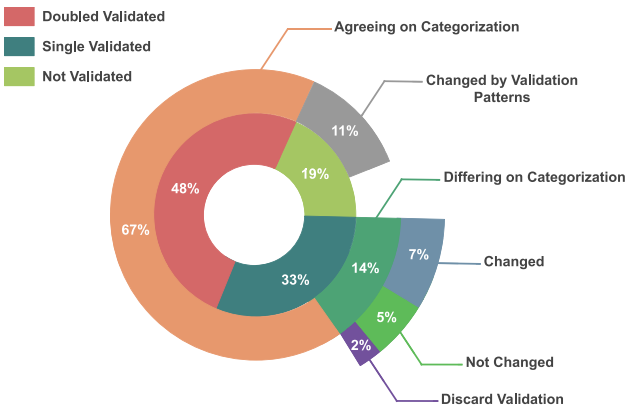


Fig. 5. Results for the validation of the categorization.

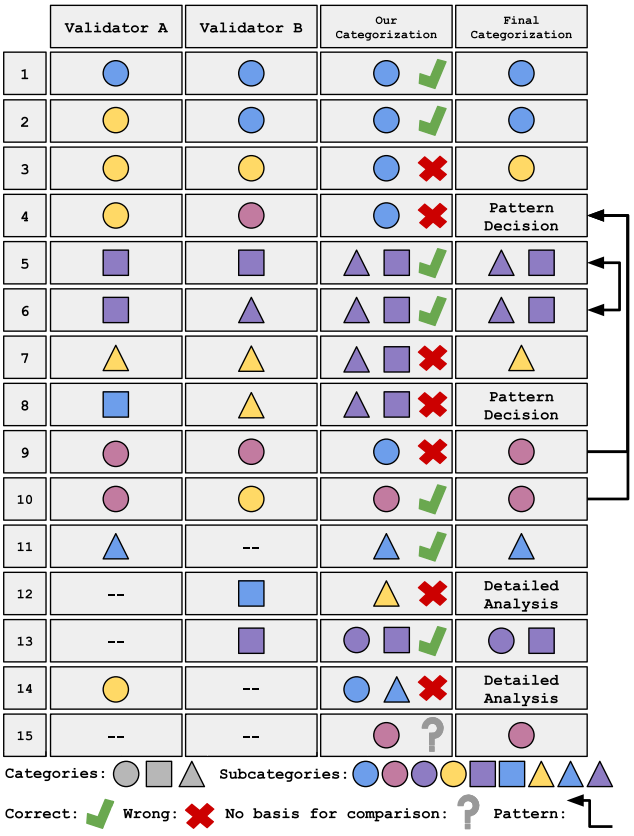


Fig. 6. The diagram visually depicts the validation cases and decision rules utilized in the categorization validation process.

subcategories, such as merging Filesystem and Resource Information Maintenance into Filesystem and Resource Management, respectively. Similar adjustments were made to four process management subcategories, refining the initial 76 subcategories to 66. These changes allowed for the validation of 92% of system calls and significantly improved the categorization.

4.3.2. Representation validation

To validate graph edges, we focused on V cost ($T/4$) and X cost ($T/8$) relationships, as class and subclass edges were already validated. A total of 282 edges were reviewed: 201 V cost and 81 X cost. Validation used 14 questionnaires with 20–21 questions each, representing relationships between two system calls. Edges were randomly assigned,

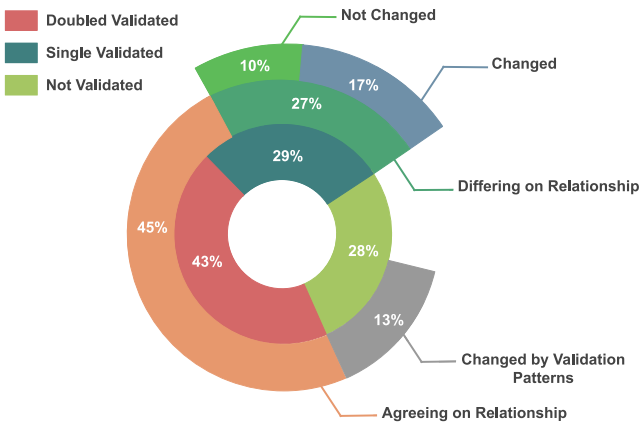


Fig. 7. Results for the validation of the relationships.

with a limit of five X cost edges per questionnaire, as X cost relationships are easier to identify due to the similarity in system call names.

In the questionnaire, each validator was presented with four system call options per question and tasked with selecting the one most closely related to the system call in the question statement. This decision was based on analyzing tasks, resources, and parameters. For example, to validate the X cost relationship between *umount*() and *umount2*(), the question asked: "*Which of the system calls below has a close relationship with umount(2)?*" Options included: the correct match (*umount2*()), a call with a V cost edge (*mount*()), a related call from the same subcategory but without a V or X cost edge (*sysfs*()), and a completely unrelated call (*seccomp*()).

For V cost edges, the same rules applied, but options with X cost relationships were excluded to avoid obvious answers. For instance, when validating the V cost relationship between *pread64*() and *preadv*(), including *preadv2*(), which has an X cost relationship with *preadv*(), was avoided to ensure a fair evaluation.

Fig. 7 presents the results of the second validation. With 16 responses, 43% of edges received a double review, and 72% were reviewed at least once. Validation compared whether the validator's selected system call matched our choice for forming an edge. Positive cases, where the validator agreed on the V or X cost categorization, accounted for 45% of edges (e.g., rows 1, 2, and 6 in Fig. 8). In 27% of cases, validators selected a system call from the same class and subclass but not our chosen edge. For these, we created a new edge and deleted the incorrect one, updating 17% of edges (e.g., rows 3 and 8). If the chosen system call did not belong to the same class or subclass, the response was considered noise, and the original edge was kept. This accounted for 10% of differing edges (e.g., rows 5 and 7).

During relationship validation, we identified patterns similar to those in the initial phase, aiding in validating unreviewed edges. One notable pattern involved set and get system calls. Initially classified with X cost relationships due to their opposite actions but shared resources, most validators found other options closer. Based on this feedback, we adjusted these edges from X cost to V cost, aligning better with validator perspectives.

We observed that validators consistently viewed reading and writing system calls as highly related, prompting us to revise their relationships from V cost to X cost edges. Following validation, the final graph had 193 V cost edges and 88 X cost edges, with 85% directly or indirectly validated. These adjustments improved the system call graph and refined our representation.

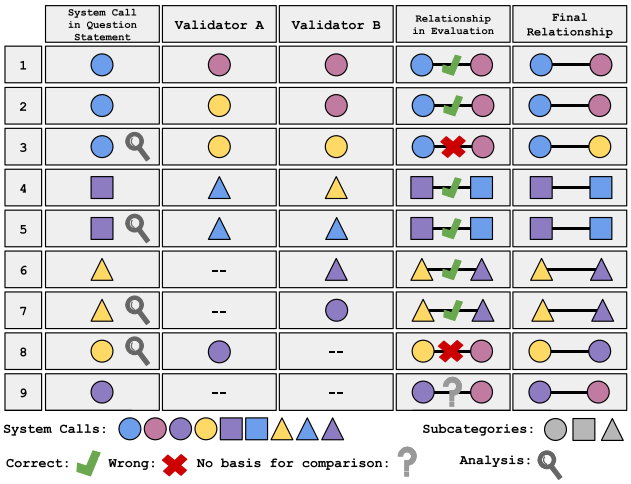


Fig. 8. The diagram visually depicts the validation cases and decision rules utilized in the relationship validation process.

5. Results and discussion

This section presents the results of the preliminary and extended studies. Due to the impracticality of showcasing the complete results for all configurations of each technique, we have opted to focus our discussion on the results from the best configuration for each technique. The best results in this case are determined based on the configuration that achieved the highest True Positive Rate (TPR) during the attack phase for the majority of vulnerabilities and the lowest False Positive Rate (FPR) during both the pre-attack and post-attack phases. Nevertheless, the comprehensive results for all configurations are available here: https://figshare.com/s/6152d90559dac565f248. Note that, the results presenetd are the mean values obtained from the tests conducted on each collection as explained in Section 3.5.

5.1. Preliminary study

Fig. 9 presents the results of the preliminary study (not using our call graph representation), divided into three phases (i.e., pre-attack, attack and post-attack). The first noticeable aspect is the difference in scales between the False Positive Rate (FPR) and the True Positive Rate (TPR), which is both normal and intentional, as the goal is to achieve a high TPR and a low FPR. In the pre-attack phase, Fig. 9(a), the FPR are relatively low for all machine learning techniques and for all vulnerabilities, with none exceeding 0.100. Elliptical Envelope shows a maximum value higher than the remaining ones (0.089 for vulnerability V1), while the others remain very low, below 0.050. These results are promising for all techniques except for the Elliptical Envelope. However, this phase alone does not provide a complete evaluation. It is essential to consider the results across all three phases to comprehensively assess a technique's effectiveness.

The analysis of the TPR during the attack phases, in Fig. 9(b), reveals more discrepant results, as most techniques were not very effective in detecting attacks through vulnerabilities V1, V2, and V3 (see Table 1 for the vulnerabilities description). In fact, for these three vulnerabilities, the TPR remained below 0.200, with the Multi-Layer Perceptron (MLP) emerging as the best technique for identifying them. On the other hand, better results were observed for V4, as most techniques performed well in detecting attacks to this vulnerability, with some achieving TPR values close to 1. For V5, although the detection results were not as high as those for V4, they were still sufficient to generate a reliable response (TPR is approximately 0.350, with some variations). This can be attributed to the understanding that not all data during the attack phase represents attack instances; normal instances may also occur during this period. Thus, even with a medium TPR, it is possible to alert the system that an attack is happening, aiding in the management of this type of data.

In the post-attack phase, one noticeable trend is that the FPR is higher compared to the pre-attack phase (as high as 0.140 in certain cases). This is expected, as after an attack, even when it has concluded, the system may take some time to return to its normal state. Consequently, some post-attack data is still influenced by the attack, which is detected by the system and reflected in a higher FPR.

An interesting observation is that the Isolation Forest technique did not perform well across any vulnerability in any of the three phases. Specifically, it exhibited higher False Positive Rates (FPR) during the pre-attack (around 0.060) and post-attack phases (around 0.080) and a lower TPR during the attack phase for V4 and V5 (0.334 and 0.216, respectively). However, it demonstrated better results than most techniques for V1, V2, and V3 during the attack phase, which could indicate a stronger capability to handle imbalanced data (these vulnerabilities are more significantly affected by such data imbalances).

In summary, the machine learning algorithms tested were able to train models that are highly effective in detecting V4 and V5 but showed limited success for V1, V2, and V3. These results do not meet the expectations for intrusion detection in containerized services. IN the next section we will examine the findings of the extended study, which incorporates our novel system call representation.

5.2. Extended study

In this section, we present the results of our evaluation using the system call graph representation across a series of tests. First, we revisit the results achieved using various ML techniques. This is followed by an analysis of a post-processing window approach. Finally, we demonstrate how anomaly-based techniques from this study can be trained and evaluated for their effectiveness in identifying zero-day attacks.

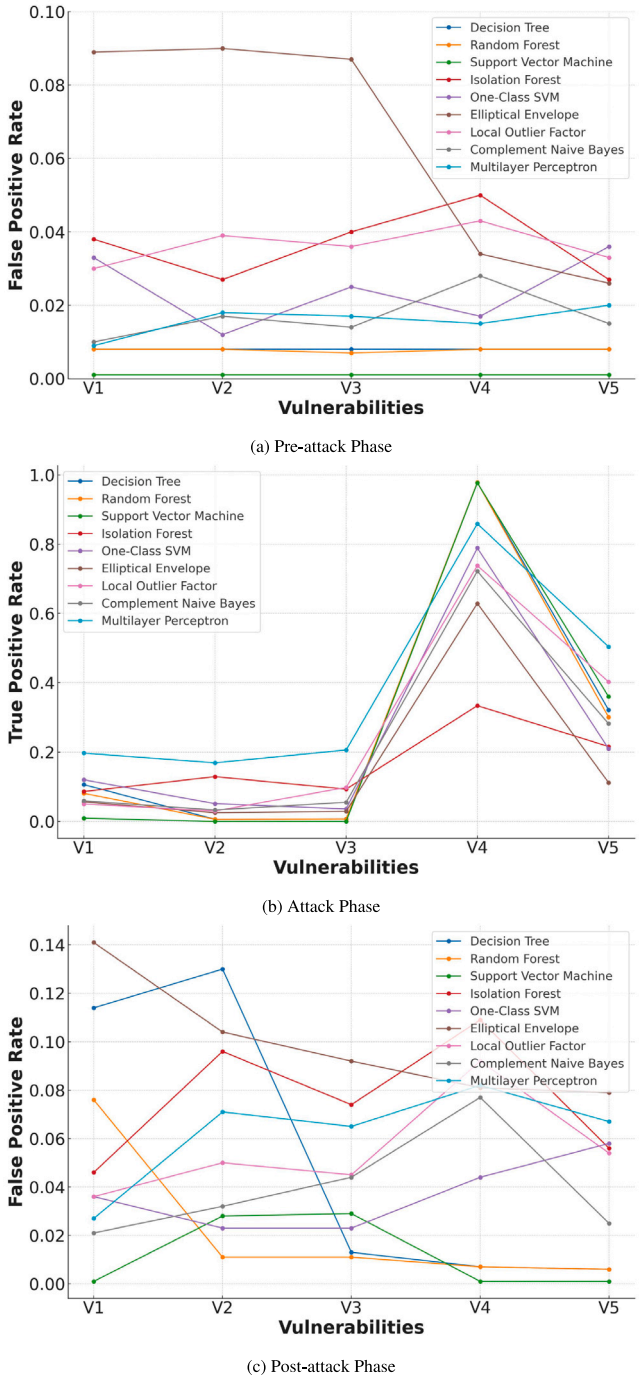
5.2.1. ML techniques evaluation

The results of our extended study are presented in Fig. 10. One of the first observations we can make is the noticeable increase in the FPR during both the pre-attack and post-attack phases (exceeding 0.100 for both phases). This increase is particularly pronounced for some techniques, such as the Elliptical Envelope, which shows significant growth compared to the preliminary study (around 0.250 in Fig. 10(a) and 0.300 in Fig. 10(c)). The rise in FPR is an expected outcome, as the extended study incorporates more information and an increased number of features due to the dimensionality of the embeddings. However, while some techniques exhibit substantial increases, others remain within a safer range, highlighting differences in their robustness.

When analyzing the attack phase in Fig. 10(b), we observe improvements compared to the preliminary study. For V1, V2, and V3, all techniques achieved higher TPR than those recorded in the preliminary study (around 0.200 higher). In most cases, this improvement was modest, but for certain techniques, such as Multilayer Perceptron, Decision Tree, and Random Forest, the increase was significant (0.293, 0.214 and 0.178, respectively for V1). This suggests that these techniques are more effective in raising alarms during an intrusion, meeting the critical requirement of detecting attacks accurately.

For V4, the results also showed improvement. However, since the preliminary study already achieved very strong results for V4, this improvement is less critical. As for V5, we observe a substantial increase in the TPR, further demonstrating the benefits of the extended study. For example, the One-Class SVM and Local Outlier Factor achieved TPRs of 0.562 and 0.548, respectively, in V5.

When we analyze the mean increase in TPR for the techniques in the extended study, we observe that all results improved by at least 193%. This is a significant improvement and highlights the potential of our



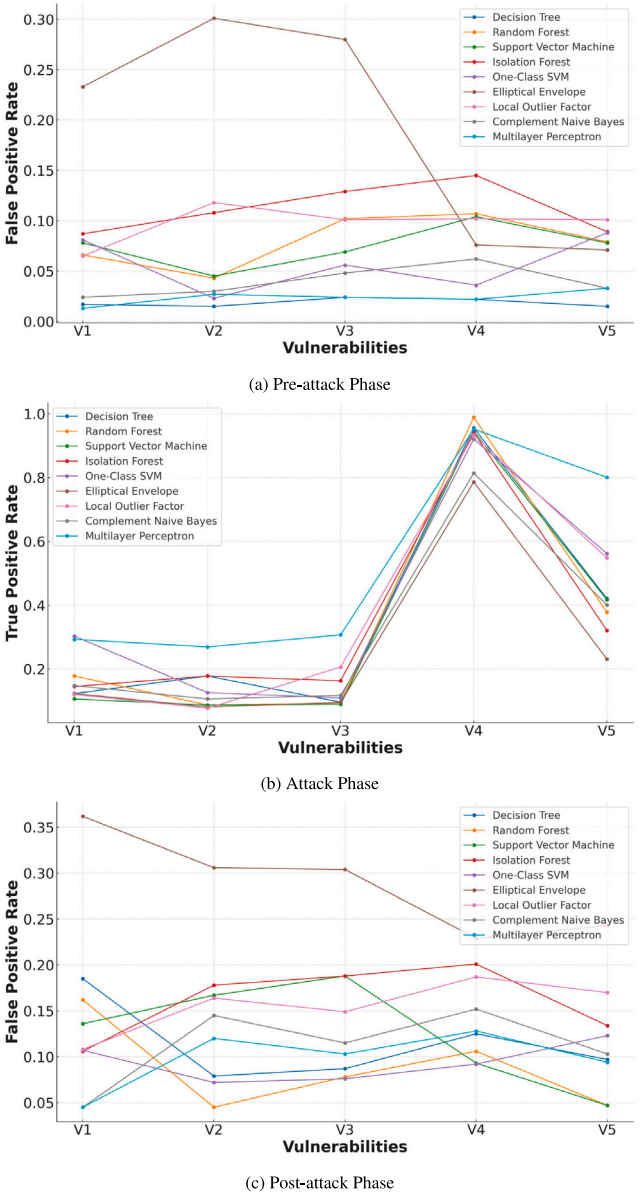
(a) Pre-attack Phase

(b) Attack Phase

(c) Post-attack Phase

Fig. 9. Results from the preliminary evaluation that uses feature vectors and label encoding representation for system calls.

representation for intrusion detection. Additionally, this extended evaluation allows for a more precise assessment of the techniques. Among them, the best results are achieved by the Multilayer Perceptron, which appears to be better equipped to handle this type of data (the TPR values for V1, V2, V3, V4, and V5 are 0.293, 0.188, 0.209, 0.952, and 0.801, respectively, as shown in Fig. 10(b)). This could indicate that neural networks are particularly suited for intrusion detection scenarios in containerized services. Further studies are necessary, including exploring other neural network architectures or configurations within the same family of techniques.



(a) Pre-attack Phase

(b) Attack Phase

(c) Post-attack Phase

Fig. 10. Results from the extended evaluation that uses our system call graph representation.

Decision Tree and Random Forest also show promise as reasonable choices based on these results. However, there is still room for improvement in identifying vulnerabilities V1, V2, and V3. A notable limitation in this evaluation stems from the imbalanced distribution of these three vulnerabilities, which account for less than 1% of the data in their respective collections. While efforts were made to minimize this imbalance through technique configurations, the current dataset imposes inherent constraints. Another potential issue is that the attack traces from these vulnerabilities may be very similar to normal traces, causing difficulties for machine learning techniques to classify them accurately. This challenge is also discussed in the evaluation of zero-day attacks. Extending the study to gather more data for these vulnerabilities or utilizing datasets with a more balanced representation would be ideal for future work.

Unfortunately, some techniques, such as Elliptical Envelope, Isolation Forest, and SVM, produced less favorable results, primarily exhibiting significantly higher FPR: around 0.250, 0.100, and 0.080 in the

pre-attack phase (Fig. 10(a)), and 0.300, 0.180, and 0.150 in the post-attack phase (Fig. 10(c)), respectively. This demonstrates that not all ML techniques are suitable for this type of evaluation and subsequent detection systems.

5.2.2. Post-processing with windows

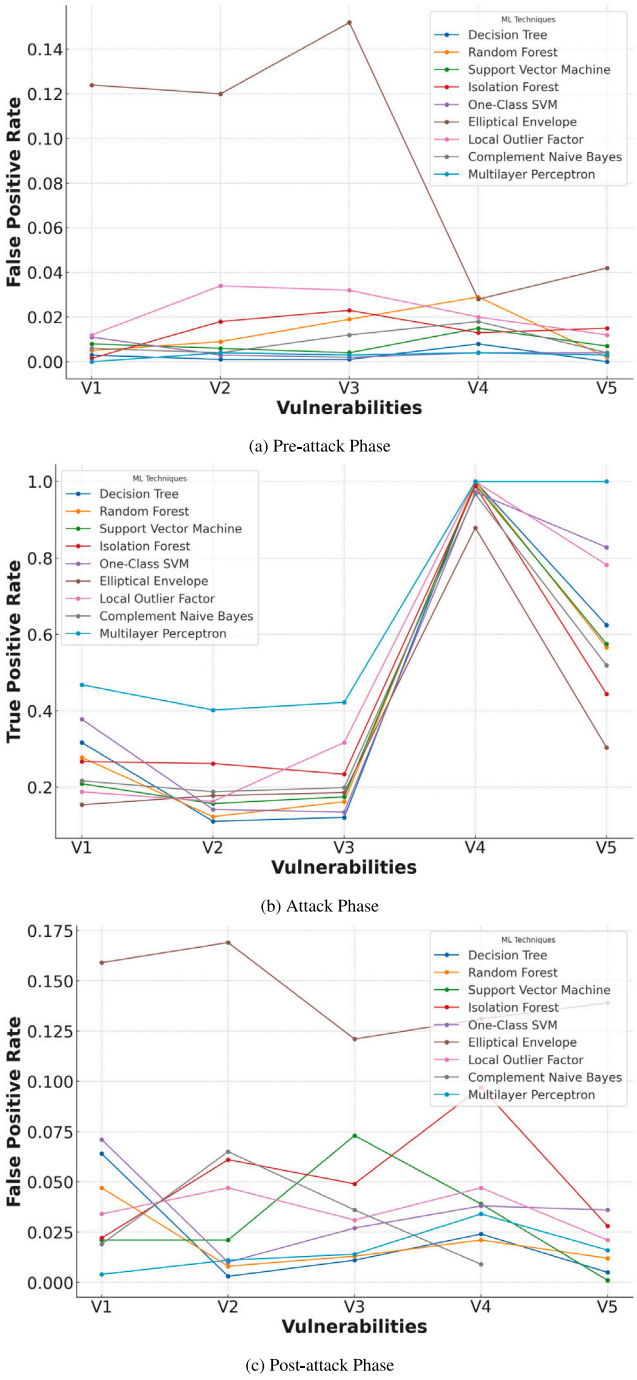
The process of post-processing aims not to enhance the detection of attacks per data sample (as this is already performed by the machine learning models), but rather to increase the alarms raised during the attack phase and minimize the false alarms triggered in the pre- and post-attack phases. In other words, we are no longer evaluating the True Positive Rate (TPR) or False Positive Rate (FPR) based on the number of instances in the test dataset; instead, TPR and FPR are now calculated based on the windows in each phase (i.e., pre-attack, attack, post-attack) of our post-processing approach. For example, if we initially classified 100 instances as attacks during our attack phase, out of a total of 1000 instances, TPR would be calculated as $100/(100+900) = 0.1$. However, for our post-processing approach, we would analyze not individual instances but rather windows. Using an overlapping sliding window with a size of 50 that shifts by 25 instances at each step, we would have a total of 39 windows in this example. Based on a threshold for detecting attacks within a window, 15 of these windows would trigger attack alerts. Consequently, the TPR in this scenario would be calculated as $15/(15+24) = 0.384$. This outlines how the TPR and FPR are calculated during post-processing evaluation. This means that comparisons between the TPR and FPR from this evaluation and those from previous evaluations should be made with caution, as the scales are different. Fig. 11 shows these results.

The first notable observation is a significant reduction in the number of alerts raised during the pre- and post-attack phases. For the pre-attack phase (Fig. 11(a)), most techniques achieve an FPR of approximately 0.040, which is remarkably low and well within an acceptable range. In the post-attack phase, while the reduction is substantial, the FPR is still higher than in the pre-attack phase (e.g., Random Forest) shows around 0.015 in pre-attack (Fig. 11(a)) and 0.030 in post-attack (Fig. 11(c)). This is likely due to the system still recovering from the attack, leading to residual activity being captured shortly after the attack. Even so, most techniques exhibit an FPR of less than 0.075 for alerts in the post-attack phase, which represents a favorable outcome. During the attack phase (Fig. 11(b)), we observe a marked increase in TPR across all techniques (e.g., for V1: MLP at 0.468, One-Class SVM at 0.378, and DT at 0.317). Notably, the multilayer perceptron effectively identifies attacks throughout the attack phase of V4 and V5 (i.e., 1.0 for both). Although the results for V1, V2, and V3 from the remaining techniques are not as high, they remain close to 0.200, which could suffice to capture the attack and activate countermeasures within the system.

This is an important consideration, as it is unrealistic to expect an intrusion detection system to identify attacks throughout their entire duration. The system only needs to detect the attack once to issue a warning and trigger the necessary countermeasures. Thus, even with a lower TPR, the system can still effectively detect intrusions. In the case of post-processing with a sliding window, a single window raising an alarm during the attack phase is sufficient to alert the system to a potential attack. However, security systems typically prefer to detect multiple consecutive windows with attack indicators before initiating countermeasures, reducing the likelihood of false alarms. The key advantage lies in even in scenarios with lower attack detection rates, significantly reduces the number of false alarms. Our evaluation demonstrates this clearly, with a substantial minimization of the FPR in both the pre-attack and post-attack phases.

Among the techniques, the multilayer perceptron demonstrates excellent adaptability to this scenario, showing significant improvement across all phases (i.e., pre-attack is below 0.010, the attack phase is around 0.400 for V1, V2, and V3, and 1.0 for V4 and V5, while post-attack is approximately 0.025). For some vulnerabilities, the One-Class SVM also exhibits notable TPR (i.e., V1 at 0.378, V4 at 0.972, and V5 at 0.828, as shown in Fig. 11(b)). Isolation Forest achieves good results during the pre-attack (around 0.015) and attack (around 0.240 for V1, V2, and V3; 0.980 for V4; and 0.450 for V5) phases but remains affected during the post-attack phase (around 0.070). Conversely, the elliptical envelope, despite showing some improvement, continues to underperform compared to its peers.

These results highlight the effectiveness of applying a sliding-window post-processing approach in conjunction with machine learning techniques. This combination offers an interesting strategy for intrusion detection, addressing some of the limitations of the techniques,



(a) Pre-attack Phase

(b) Attack Phase

(c) Post-attack Phase

Fig. 11. Results from the Post-processing sliding window approach. Window size = 50 and Threshold = 10%.

particularly in reducing false alarms during the pre- and post-attack phases.

5.2.3. Zero-day attacks

Although intrusion detection systems can leverage high-quality datasets that accurately represent real-world threats, it is impossible for these datasets to encompass every potential threat a system might face. This limitation arises because new vulnerabilities and attack methods can emerge without prior knowledge from the security community. As a result, intrusion detection systems must be robust enough to handle attacks they have not been trained on or were not designed to expect. These types of attacks are known as zero-day attacks, which exploit undisclosed vulnerabilities or utilize novel techniques that are either unprecedented or uncommon (Guo, 2023).

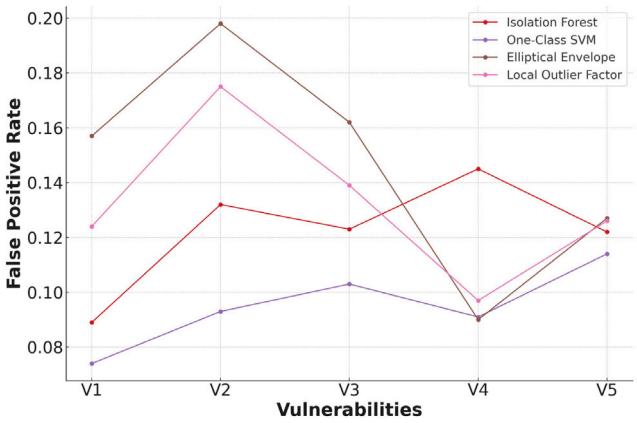
Identifying zero-day attacks is a challenging task. However, machine learning offers significant advantages in combating such threats through anomaly detection approaches. In anomaly detection, the model is typically trained to represent the system's normal execution behavior. Any deviation from this learned norm, such as an attack, is recognized as an outlier, allowing the system to flag it as anomalous activity (Nassif et al., 2021).

To evaluate this scenario, we chose to use only the anomaly detection techniques from our selected machine learning methods: Isolation Forest, One-Class SVM, Elliptic Envelope, and Local Outlier Factor. This decision was based on the need to assess how these models would respond to previously unseen attacks. For training, we utilized our large collection of benign data for each anomaly detection technique. We also applied our system call graph representation to this evaluation.

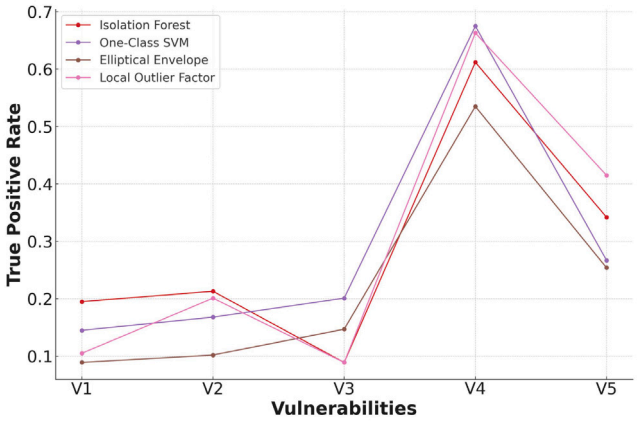
In this testing phase, we employed the default configuration for all the selected techniques. Subsequently, we tested the models against each of the five vulnerabilities, analyzing the three collections of data separately for each vulnerability. The results presented here represent the mean performance across the three collections for each vulnerability.

Based on the results shown in Fig. 12, it is evident that anomaly-based techniques can identify attacks even in the absence of characteristic attack traces specific to the vulnerabilities. However, these techniques follow a pattern consistent with the findings from the extended study: vulnerabilities V1, V2, and V3 exhibit significantly lower TPR during the attack phase (around 0.150) compared to V4 and V5 (around 0.600 and 0.350, respectively) as shown by Fig. 12(b). This discrepancy may indicate insufficient data to create robust classifications for these attacks using the first three vulnerabilities. Alternatively, this may suggest that the attacks associated with V1, V2, and V3 closely resemble normal system behavior, making them challenging to detect as outliers. This is supported by Table 2, which shows that V5 is also highly imbalanced, similar to these vulnerabilities. However, V5 is still well-identified, indicating that detecting attack traces from V5 is easier than from V1, V2, and V3, even in an imbalanced scenario. This further suggests the possibility that the attack traces for V1, V2, and V3 are very similar to normal traces, contributing to the difficulty in detection.

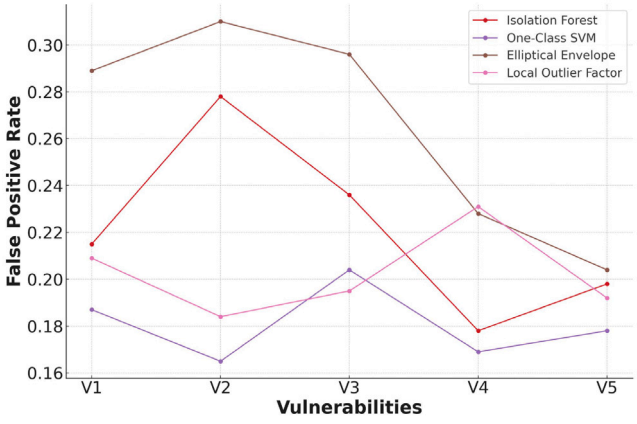
The FPR values increases during both pre-attack (around 0.140 in Fig. 12(a)) and post-attack phases (around 0.230 in Fig. 12(c)), which is undesirable for an effective detection system. Our post-processing evaluation suggests that integrating a sliding window post-processing approach could improve detection accuracy and mitigate this issue. Another notable observation is that the FPR for V4 and V5 is consistently lower than that of the other vulnerabilities in at least three techniques (i.e., only IF with 0.145 in Fig. 12(a) for V3 and LOF with 0.230 in Fig. 12(c) for V3). This indicates a higher precision in identifying anomalies for these two vulnerabilities.



(a) Pre-attack Phase



(b) Attack Phase



(c) Post-attack Phase

Fig. 12. Results from the Zero-day evaluation.

5.3. Comparative analysis

Based on the results from both our preliminary and extended studies, we have identified improvements in the field by analyzing a broader range of techniques compared to previous research, such as Cavalcanti et al. (2021), El Khairi et al. (2022) and Rocha et al. (2022). Our analysis demonstrates that the use of system call graph representation as an embedding method for categorical data has not been previously explored on such a wide scale. Existing works that employ graph representations for system calls are typically tailored to specific scenarios, such as Android malware detection or Security Operations Centers (SOCs). In contrast, our approach is more generalized and can

be applied beyond containerized environments, as long as Linux system calls are utilized.

When examining the work of Flora et al. (2020), which serves as a foundation and shares the same datasets, we observe improvements in intrusion detection performance, particularly in reducing false alarms. For V1, V2, and V3, the results fall below those of non-ML techniques used in Flora et al. (2020). However, as previously demonstrated, our method fulfills its purpose as long as it can trigger alarms and initiate countermeasures, even if the True Positive Rate (TPR) during attacks is lower.

For V4 and V5, our results are comparable, with V4 consistently achieving above 0.9 TPR and V5 exceeding 0.5, reaching levels similar to or surpassing those in Flora et al. (2020). The most notable improvement lies in the False Positive Rate (FPR). In our work, the FPR remains at an optimal threshold, averaging around 0.2, whereas in Flora et al. (2020), this average increases to 0.3. Moreover, our post-processing window further reduces the FPR to below 0.1.

These findings suggest that our approach offers distinct advantages in some areas while highlighting aspects that require further exploration. Future work should investigate additional solutions or complementary approaches that can enhance our system call representation and improve overall performance.

6. Threats to validity

The current work is susceptible to a variety of threats that may affect its validity. These threats can be categorized into two distinct aspects: those related to the evaluation of the ML models and those concerning the system call graph-based representation. To address these issues effectively, next we analyze each aspect individually, providing a clearer understanding of their potential impact and implications.

6.1. System call representation

A key threat to the enhanced system call representation is the inherent subjectivity in its development. Researchers defined categorization and relationships, and while steps such as involving multiple reviewers and resolving conflicts were taken, complete objectivity is unattainable. Validation processes were implemented to reduce subjectivity by independently assessing classifications and relationships. However, limited responses restricted double-reviewing all steps. Expanding the pool of validators could address this in the future.

Among the two validations conducted, the validation of system call relationships and the creation of graph edges proved more constrained. Validators were not allowed to suggest new connections or patterns, limiting the discovery of novel relationships. Future validations could incorporate mechanisms to allow broader input and enable new insights.

Lastly, system calls themselves are dynamic. Changes to the Linux Kernel, including new system calls, deprecations, or modifications to tasks and parameters, may impact classifications and relationships. These updates could lead to the reassignment of system calls or the need to redefine their connections in the graph. Given this evolving nature, our representation must remain adaptable, requiring periodic updates and refinements.

6.2. ML evaluation

In our evaluation, although we incorporated a broad range of ML techniques, there remains room for further exploration, particularly in the area of neural networks. The promising performance of Multilayer Perceptrons suggests that additional neural network architectures could yield even better results. Moreover, experimenting with new configurations for the existing techniques could lead to improved outcomes, though such efforts require significant time and effort.

Another limitation lies in the dataset used. While we tested the dataset extensively, it has reached its practical limits for analysis. Issues related to data imbalance have significantly affected performance, particularly in improving the True Positive Rate (TPR). Testing with alternative datasets would be ideal to determine if the challenges in enhancing TPR are inherently tied to the imbalance in the current dataset. Furthermore, while the dataset includes attacks to five vulnerabilities, this represents only a small subset of the possible attack vectors. Although the techniques demonstrated resilience against zero-day attacks, testing with other vulnerabilities is crucial to evaluate the representation's adaptability across diverse scenarios.

Regarding the dataset, one potential issue is its relevance given the time of data collection and the versions of the systems used. The data was collected based on system calls retrieved from the containerization of an older version of MariaDB (v5.5.28) and Ubuntu 14.04 LTS. This setup was chosen to ensure that the system was as representative as possible of real-world scenarios, with well-documented attacks and available proof-of-concept (PoC) exploits. Although the data remains relevant for this study, it is important to acknowledge that these system versions are outdated. Testing with more up-to-date data and setups would provide valuable insights into the impact of newer technologies and evolving attack techniques. This is one of the key goals for the continuation of this project, as evaluating a broader range of vulnerabilities and datasets is essential for improving generalization and robustness.

Regarding our preliminary study, we need to briefly discuss the number of system calls per instance. This number was determined based on a trade-off between achieving the best results during the attack phase while minimizing false alarms. For the dataset used, an instance size of 50 was found to be optimal, yielding the best results. However, this value may vary for different datasets, requiring an evaluation of the appropriate size for any attempt to replicate the process with a different dataset. Failure to carefully analyze this factor could impact the correlation between studies and lead to different results

The dimensionality of graph embeddings significantly impacts the computational resources required by machine learning techniques. Higher dimensionality increases both resource consumption and processing time. While adjusting dimensionality, either increasing or decreasing, can lead to notable differences in detection results, it is essential to evaluate higher or lower dimensions and their results to determine an optimal balance. This involves finding a cost-effective trade-off between the embedding dimensionality for system call representation and the resource expenditure required for processing.

One interesting observation is that the selected vulnerabilities were more effectively identified by anomaly-based methods. This may indicate that, at least for Vulnerabilities V4 and V5, the system call traces of these attacks are highly distinguishable from normal execution. However, for V1, V2, and V3, this distinction is less clear. One possible reason for their lower TPR could be that their attack traces more closely resemble normal system execution. This suggests that anomaly-based intrusion detection methods may be less effective for attacks that closely mimic normal behavior. In such cases, techniques like post-processing, as used in our study, or even pre-processing methods could be valuable in improving detection rates. However, further research is needed to provide a definitive answer on the best approach for handling such scenarios.

Lastly, our evaluation focused primarily on the effectiveness of the proposed solutions for intrusion detection in containerized services. However, operational considerations, such as resource usage and model training time, were not accounted for. These factors are critical for real-world applications and should be addressed in future studies to ensure a comprehensive evaluation of the proposed approaches.

7. Conclusion

Evaluating machine learning techniques for intrusion detection is challenging due to the multitude of techniques, configurations, and

data processing methods available. This work specifically focused on understanding how this evaluation can be conducted and identifying the best ML-based approach in the context of containerized services. Our evaluation demonstrated the suitability of certain techniques for these scenarios and highlighted the critical role of data representation.

We successfully developed, validated, and tested a novel system call representation based on graph theory, which preserves inherent information about system calls and their relationships. While the results are not extensively groundbreaking, they show measurable improvements using this representation and outline steps for further enhancement. Additionally, we verified the effectiveness of post-processing using a sliding window approach in improving the TPR while minimizing false alarms in our scenario. By exploring the feasibility of detecting zero-day attacks, we also advanced the understanding of machine learning techniques that are better suited for robust intrusion detection.

Our next steps include refining the system call representation to incorporate any newly introduced or deprecated system calls, as well as updated documentation. This refinement will enhance validation and strengthen the overall framework. Also, the evaluation of system call parameters and their potential as crucial information for enhancing intrusion detection is also analyzed. We also plan to continue exploring machine learning techniques, with a specific focus on neural networks, which appear particularly promising in this scenario. However, we recognize that the dataset used in this study has reached its limitations. Expanding our evaluation to new datasets will be a key priority, enabling us to test the scalability and applicability of our methods in broader contexts. Additionally, we are working on constructing a new dataset of system calls for security scenarios as a means to further validate our representation and provide more data to the community.

CRediT authorship contribution statement

Iury Araujo: Writing – review & editing, Writing – original draft, Visualization, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Marco Vieira:** Writing – review & editing, Validation, Supervision, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially funded by Fundação para a Ciência e Tecnologia (FCT) grant 2022.11551.BD. It is also partially financed through national funds by FCT - Fundação para a Ciência e a Tecnologia, I.P., in the framework of the Project UIDB/00326/2025 and UIDP/00326/2025. Content produced within the scope of the Agenda "NEXUS - Pacto de Inovação - Transição Verde e Digital para Transportes, Logística e Mobilidade", financed by the Portuguese Recovery and Resilience Plan (PRR), with no. C645112083-00000059 (investment project no. .° 53)

Data availability

Data will be made available on request.

References

Abdulganiyu, O.H., Ait Tchakoucht, T., Saheed, Y.K., 2023. A systematic literature review for network intrusion detection system (IDS). Int. J. Inf. Secur. 22 (5), 1125–1162.

Abed, A.S., Clancy, T.C., Levy, D.S., 2015. Applying bag of system calls for anomalous behavior detection of applications in linux containers. In: 2015 IEEE Globecom Workshops. GC Wkshps, pp. 1–5. http://dx.doi.org/10.1109/GLOCOMW.2015.7414047.

Aghaei, E., Serpen, G., 2019. Host-based anomaly detection using eigentraces feature extraction and one-class classification on system call trace data. arXiv preprint arXiv:1911.11284.

Araujo, I., Antunes, N., Vieira, M., 2023. Evaluation of machine learning for intrusion detection in microservice applications. In: Proceedings of the 12th Latin-American Symposium on Dependable and Secure Computing. pp. 126–135.

Bagherzadeh, M., Kahani, N., Bezemer, C.-P., Hassan, A.E., Dingel, J., Cordy, J.R., 2018. Analyzing a decade of linux system calls. Empir. Softw. Eng. 23, 1519–1551.

Balyan, A.K., Trivedi, N.K., Sharma, S.K., 2023. A survey on machine learning techniques for detecting, mitigating, and preventing intrusions. In: 2023 3rd International Conference on Technological Advancements in Computational Sciences. ICTACS, IEEE, pp. 1368–1375.

Canfora, G., Medvet, E., Mercaldo, F., Visaggio, C.A., 2015. Detecting android malware using sequences of system calls. In: Proceedings of the 3rd International Workshop on Software Development Lifecycle for Mobile. pp. 13–20.

Castanhel, G.R., Heinrich, T., Ceschin, F., Maziero, C., 2021. Taking a peek: An evaluation of anomaly detection using system calls for containers. In: 2021 IEEE Symposium on Computers and Communications. ISCC, IEEE, pp. 1–6.

Cavalcanti, M., Inacio, P., Freire, M., 2021. Performance evaluation of container-level anomaly-based intrusion detection systems for multi-tenant applications using machine learning algorithms. In: Proceedings of the 16th International Conference on Availability, Reliability and Security. pp. 1–9.

El Khairi, A., Caselli, M., Knierim, C., Peter, A., Continella, A., 2022. Contextualizing system calls in containers for anomaly-based intrusion detection. In: Proceedings of the 2022 on Cloud Computing Security Workshop. pp. 9–21.

Flora, J., Gonçalves, P., Antunes, N., 2020. Using attack injection to evaluate intrusion detection effectiveness in container-based systems. In: 2020 IEEE 25th Pacific Rim International Symposium on Dependable Computing. PRDC, IEEE, pp. 60–69.

Forrest, S., Hofmeyr, S., Somayaji, A., 2008. The evolution of system-call monitoring. In: 2008 Annual Computer Security Applications Conference (Acsac). IEEE, pp. 418–430.

Grimmer, M., Röhling, M.M., Kricke, M., Franczyk, B., Rahm, E., 2018. Intrusion detection on system call graphs. Sicherh. Vernetzten Syst. G1– G18.

Grover, A., Leskovec, J., 2016. Node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 855–864.

Guo, Y., 2023. A review of machine learning-based zero-day attack detection: Challenges and future directions. Comput. Commun. 198, 175–185.

Hamid, Y., Sugumaran, M., Journaux, L., 2016. Machine learning techniques for intrusion detection: a comparative analysis. In: Proceedings of the International Conference on Informatics and Analytics. pp. 1–6.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Adv. Neural Inf. Process. Syst. 30.

Hancock, J.T., Khoshgoftaar, T.M., 2020. Survey on categorical data for neural networks. J. Big Data 7 (1), 28.

Heidari, A., Jabraeil Jamali, M.A., 2023. Internet of things intrusion detection systems: A comprehensive review and future directions. Clust. Comput. 26 (6), 3753–3780.

Hofmeyr, S.A., Forrest, S., Somayaji, A., 1998. Intrusion detection using sequences of system calls. J. Comput. Secur. 6 (3), 151–180.

Iacovazzi, A., Raza, S., 2022. Ensemble of random and isolation forests for graph-based intrusion detection in containers. In: 2022 IEEE International Conference on Cyber Security and Resilience. CSR, IEEE, pp. 30–37.

Jang, J.-w., Woo, J., Mohaisen, A., Yun, J., Kim, H.K., 2015. Mal-netminer: Malware classification approach based on social network analysis of system call graph. Math. Probl. Eng. 2015.

Joraviya, N., Gohil, B.N., Rao, U.P., 2024. DL-HIDS: Deep learning-based host intrusion detection system using system calls-to-image for containerized cloud environment. J. Supercomput. 1–29.

Kerman, A., Borchert, O., Rose, S., Tan, A., et al., 2020. Implementing a zero trust architecture. Natl. Inst. Stand. Technol. 2020, 17–17.

Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J., 2019. Survey of intrusion detection systems: Techniques, datasets and challenges. Cybersecurity 2 (1), 1–22.

Kim, G., Yi, H., Lee, J., Paek, Y., Yoon, S., 2016. LSTM-based system-call language modeling and robust ensemble method for designing host-based intrusion detection systems. arXiv preprint arXiv:1611.01726.

Kiran, A., Prakash, S.W., Kumar, B.A., Likhitha, Sameeratmaja, T., Charan, U.S.S.R., 2023. Intrusion detection system using machine learning. In: 2023 International Conference on Computer Communication and Informatics. ICCCI, pp. 1–4. http://dx.doi.org/10.1109/ICCCI56745.2023.10128363.

Laszka, A., Abbas, W., Sastry, S.S., Vorobeychik, Y., Koutsoukos, X., 2016. Optimal thresholds for intrusion detection systems. In: Proceedings of the Symposium and Bootcamp on the Science of Security. pp. 72–81.

Liu, M., Xue, Z., Xu, X., Zhong, C., Chen, J., 2018. Host-based intrusion detection system with system calls: Review and future trends. ACM Comput. Surv. 51 (5), 1–36.

Mauerer, W., 2010. Professional Linux Kernel Architecture. John Wiley & Sons.

Nassif, A.B., Talib, M.A., Nasir, Q., Dakalbab, F.M., 2021. Machine learning for anomaly detection: A systematic review. IEEE Access 9, 78658–78700. http://dx.doi.org/10.1109/ACCESS.2021.3083060.

Perozzi, B., Al-Rfou, R., Skiena, S., 2014. Deepwalk: Online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 701–710.

Ribeiro, L.F., Saverese, P.H., Figueiredo, D.R., 2017. Struc2vec: Learning node representations from structural identity. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 385–394.

Rocha, S.L., Nze, G.D.A., de Mendonça, F.L.L., 2022. Intrusion detection in container orchestration clusters: A framework proposal based on real-time system call analysis with machine learning for anomaly detection. In: 2022 17th Iberian Conference on Information Systems and Technologies. CISTI, IEEE, pp. 1–4.

Sarker, I.H., Kayes, A., Badsha, S., Alqahtani, H., Watters, P., Ng, A., 2020. Cybersecurity data science: An overview from machine learning perspective. J. Big Data 7, 1–29.

Sharma, B., Sharma, L., Lal, C., 2019. Anomaly detection techniques using deep learning in iot: A survey. In: 2019 International Conference on Computational Intelligence and Knowledge Economy. ICCIKE, IEEE, pp. 146–149.

Silberschatz, A., Galvin, P.B., Gagne, G., 2018. Operating System Concepts, tenth ed. Wiley, URL http://os-book.com/OS10/index.html.

Sreelakshmi, S., Babu, A.A., Lakshmipriya, C., Anto Gracious, L., Nalini, M., Siva Subramanian, R., 2024. Enhancing intrusion detection systems with machine learning. In: 2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems. ICSSAS, pp. 557–564. http://dx.doi.org/10.1109/ICSSAS64001.2024.10760341.

Stewart, D., Kolajo, T., Daramola, O., 2024. Machine learning for intrusion detection systems: A systematic literature review. In: Proceedings of the Future Technologies Conference. Springer, pp. 623–638.

Surendran, R., Thomas, T., Emmanuel, S., 2020. Gsdroid: Graph signal based compact feature representation for android malware detection. Expert Syst. Appl. 159, 113581.

Tripathy, S.S., Behera, B., 2023. Performance evaluation of machine learning algorithms for intrusion detection system. arXiv preprint arXiv:2310.00594.

Wang, H., Barriga, L., Vahidi, A., Raza, S., 2019. Machine learning for security at the IoT edge-a feasibility study. In: 2019 IEEE 16th International Conference on Mobile Ad Hoc and Sensor Systems Workshops. MASSW, IEEE, pp. 7–12.

Wunderlich, S., Ring, M., Landes, D., Hotho, A., 2020. Comparison of system call representations for intrusion detection. In: International Joint Conference: 12th International Conference on Computational Intelligence in Security for Information Systems (CISIS 2019) and 10th International Conference on EUropean Transnational Education (ICEUTE 2019) Seville, Spain, May 13th-15th, 2019 Proceedings 12. Springer, pp. 14–24.